Experiment No.4
Design & Implementation of Pass 2 of Two Pass Macro Processor
Date of Performance:30/03/26
Date of Submission:06/04/26

Aim: Design & Implementation of Pass 2 of Two Pass Macro Processor.

Objective: To study and implement Pass 2 of two pass Macro Processor for IBM 360 Machine.

Theory:

Macro: A macro is a unit of specification for program generation through expansion. A macro instruction is a notational convenience for the programmer. It allows the programmer to write shorthand version of a program (module programming).

Macro Processor: The macro processor replaces each macro invocation with the corresponding sequence of statements.

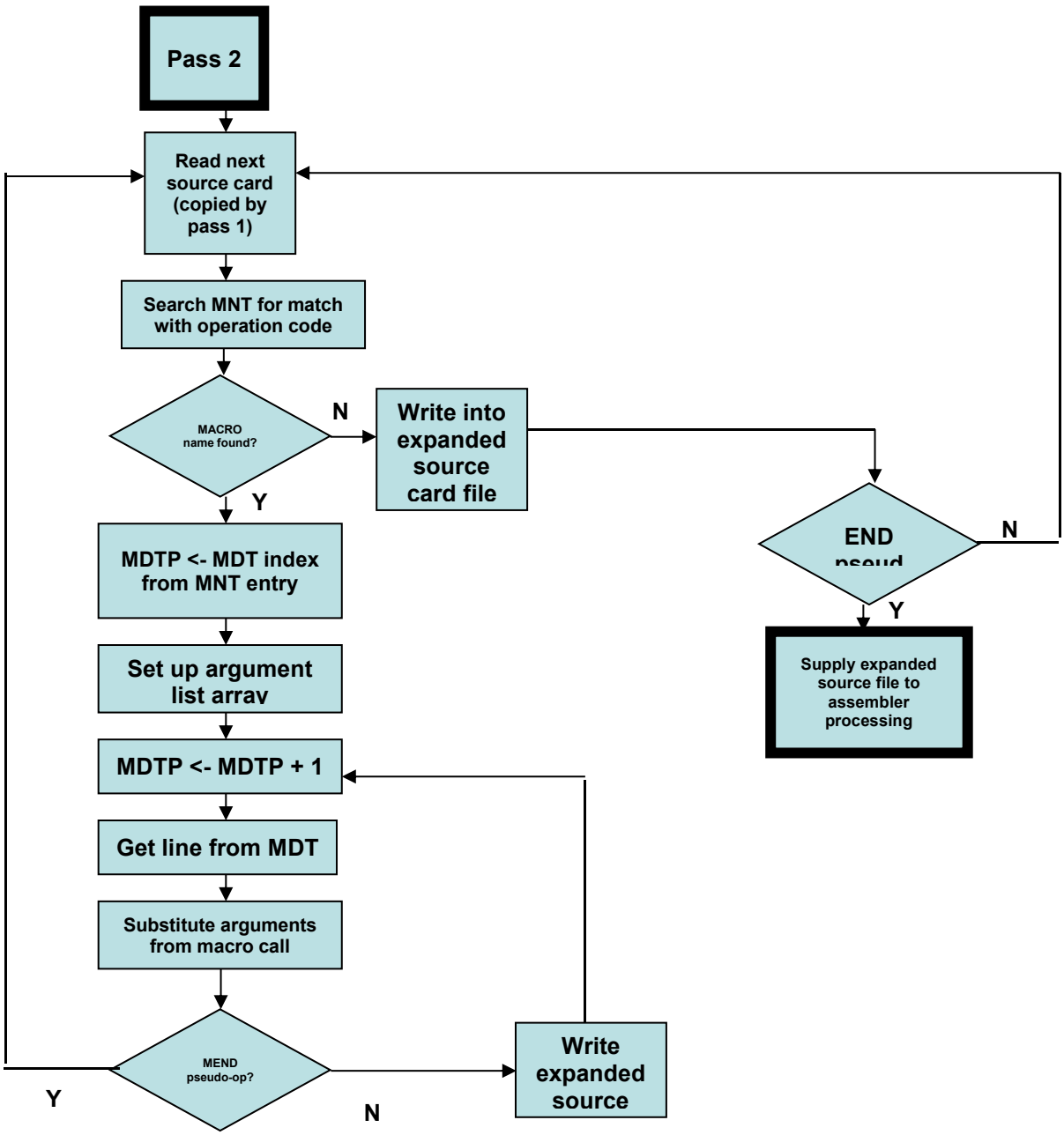
Pass 2: Expand all macro invocation statements.

- Monitors the values of expansion time variables & sequencing symbols specified in a Macro
- Handles expansion time control flow & performs expansion of model statements.

Pass 2 Databases of Macro Processor

1. Copy of source program statements
2. Output expanded source listings for input to assembler
3. MNT and MDT generated in pass 1
4. Macro Definition Table Pointer (MDTP) directs to the next statement to be used during expansion
5. An array called Parameter List Array (PLA) for substituting parameters of macro call in the source macro of the macro call in the stored macro definitions for the index correspondingly.

Pass 2 flowchart:



Code:

```
#include <stdio.h>
#include <string.h>

struct MNT
{
    char name[20];
    int mdt_index;
};

struct MDT
{
    char line[80];
};

int main()
{
    struct MNT mnt[10];
    struct MDT mdt[100];

    int mntc, mdtc;
    int i,j;

    char program[50][80];
    char word[20];

    printf("Enter number of macros: ");
    scanf("%d",&mntc);

    printf("Enter MNT entries (name index):\n");

    for(i=0;i<mntc;i++)
```

```
    scanf("%s %d",mnt[i].name,&mnt[i].mdt_index);

printf("Enter number of MDT lines: ");
scanf("%d",&mdtc);
getchar();

printf("Enter MDT lines:\n");

for(i=0;i<mdtc;i++)
    fgets(mdt[i].line,80,stdin);

int n;

printf("Enter number of program lines: ");
scanf("%d",&n);
getchar();

printf("Enter program:\n");

for(i=0;i<n;i++)
    fgets(program[i],80,stdin);

printf("\nExpanded Program:\n");

for(i=0;i<n;i++)
{
    sscanf(program[i],"%s",word);

    int found=-1;

    for(j=0;j<mntc;j++)
    {
        if(strcmp(word,mnt[j].name)==0)
```

```
        {
            found=j;
            break;
        }
    }

    if(found!=-1)
    {
        int index=mnt[found].mdt_index+1;

        while(strncmp(mdt[index].line,"MEND",4)!=0)
        {
            printf("%s",mdt[index].line);
            index++;
        }
    }
    else
    {
        printf("%s",program[i]);
    }
  }

  return 0;
}
```

Output:

```
Enter number of macros: 1
Enter MNT entries (name index):
INCR 0
Enter number of MDT lines: 5
Enter MDT lines:
INCR &A,&B
LDA &A
ADD &B
STA &A
MEND
Enter number of program lines: 5
Enter program:
START
INCR X,Y
INCR A,B
INCR P,Q
END

Expanded Program:
START
LDA &A
ADD &B
STA &A
LDA &A
ADD &B
STA &A
LDA &A
ADD &B
STA &A
END
```

Conclusion:

1. Comment on working of Program by providing input source program.
2. Comment on output generated during processing macro invocation.

Comment on working of Program by providing input source program.

The program implements a two-pass macro processor. In Pass-1, the program scans the source program and identifies macro definitions using the MACRO and MEND pseudo-operations. It stores the macro name in the Macro Name Table (MNT) and the macro body in the Macro Definition Table (MDT), while the parameters are stored in the Argument List Array (ALA). For example, if the source program contains a macro definition like MACRO INCR &A,&B, the processor stores the macro name and its body instructions such as LDA &A, ADD &B, and STA &A in the MDT. In Pass-2, when a macro call such as INCR X,Y is encountered, the processor searches the macro name in the MNT and expands the corresponding macro body from the MDT to generate the final program.

Comment on output generated during processing macro invocation.

During macro invocation processing, the macro processor replaces each macro call with the corresponding sequence of instructions stored in the Macro Definition Table. The actual parameters provided in the macro call are substituted in place of the formal parameters defined in the macro definition. For example, if the macro INCR &A,&B is invoked as INCR X,Y, the processor expands it to instructions like LDA X, ADD Y, and STA X. If the macro is called multiple times, the expansion is repeated for each invocation. As a result, the final output program contains the expanded instructions instead of the macro calls, making it ready for further processing by the assembler.